

FILA DE PRIORIDADE E MAPS

Prof^a Gláucya Boechat

glaucya.boechat@ba.docente.senai.br

Slides adaptados dos prof Rubisley Lemes
(GRUPRO)



FILA DE PRIORIDADE

3 grupo

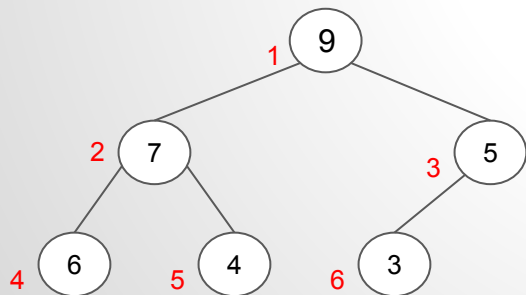
Fila de prioridade

- Estrutura de dados onde cada elemento possui uma prioridade associada a ele.
- Os elementos estão ordenados por prioridade decrescente.
- Operações básicas:
 - Inserir um novo elemento da fila de prioridade
 - Remover o elemento com a mais alta prioridade (Primeiro elemento).
- A fila de prioridade pode ser implementada com um heap (máximo)

Fila de prioridade (heap máximo)

- O heap máximo é uma árvore binária onde:
 - Onde os filhos possuem prioridade menores ou iguais a prioridade do pai
 - E a raiz do heap possui a maior prioridade

Representação em árvore



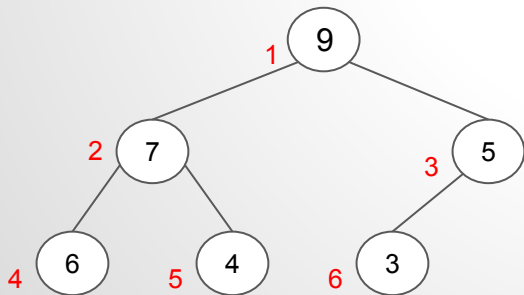
Representação em vetor



Fila de prioridade (heap máximo)

- Propriedade do Heap
 - O pai do nó k é o nó $\left\lfloor \frac{k}{2} \right\rfloor$
 - O filho à esquerda do nó k é o nó $2k$
 - O filho à direita do nó k é o nó $(2k + 1)$

Representação em árvore



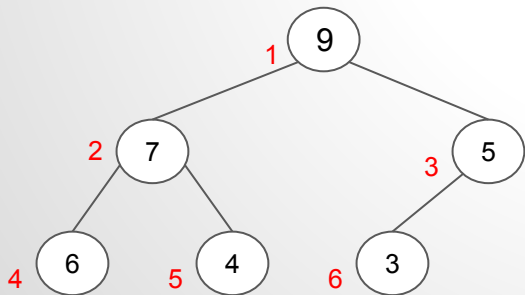
Representação em vetor



Fila de prioridade (heap máximo)

- Propriedade do Heap
 - Após as operações de remoção e inserção é preciso reorganizar o heap de acordo com as prioridades.

Representação em árvore



Representação em vetor



Fila de prioridade (heap máximo)

- Exemplos de Fila de prioridade
 - Uma fila de pacientes esperando transplante de fígado em geral é uma fila de prioridade.
 - Em Sistemas Operacionais, um exemplo é a fila de prioridade de processos aguardando o processador para execução.
 - Os processos mais prioritários são executados antes dos outros

Fila de prioridade - priority_queue

```
#include <iostream>
```

```
#include <queue>
```

```
// priority_queue -> std
```

```
using namespace std;
```

```
int main() {
```

```
    priority_queue<int> pq;
```

```
    int i, j;
```

```
    for(i=0; i < 10; i++) {
```

```
        cin >> j;
```

```
        pq.push(j);
```

```
    }
```

```
    // imprime em ordem decrescente
```

```
    while(!pq.empty()) {
```

```
        cout << pq.top() << endl;
```

```
        pq.pop();
```

```
    }
```

```
}
```

Entrada:

5

8

9

1

6

2

10

7

4

3

Saída:

10

9

8

7

6

5

4

3

2

1

Fila de prioridade - priority_queue

`#include <queue>`

Biblioteca necessária para o uso de fila de prioridade.

`priority_queue<TIPO> pq;`

TIPO é qualquer tipo da linguagem: int, char, double, vector<float>, ...

`S.push(ELEMENTO);`

Adiciona ELEMENTO do tipo TIPO na Fila de Prioridade.

`S.top();`

Retorna o elemento com maior prioridade na Fila de Prioridade.

`S.pop();`

Remove o elemento com maior prioridade da Fila de Prioridade.

`S.empty();`

Retorna verdadeiro se a Fila de Prioridade estiver vazia, e falso caso contrário.

`S.size();`

Retorna o tamanho da Fila de Prioridade

Fila de prioridade com **Heap Mínimo**

```
#include <iostream>
```

```
#include <queue>
```

```
using namespace std; // priority_queue -> std
```

```
int main() {
```

```
    priority_queue<int , vector<int>, greater<int> > pq;
```

```
    int i, j;
```

```
    for(i=0; i < 10; i++) {
```

```
        cin >> j;
```

```
        pq.push(j);
```

```
    }
```

```
    // imprime em ordem crescente
```

```
    while(!pq.empty()) {
```

```
        cout << pq.top() << endl;
```

```
        pq.pop();
```

```
    }
```

```
}
```

Entrada:

5

8

9

1

6

2

10

7

4

3

Saída:

1

2

3

4

5

6

7

8

9

10

Fila de prioridade - par de dados

```
#include <iostream>
#include <queue>
using namespace std; // priority_queue -> std

int main() {
    priority_queue<pair<int, int>> pq;
    int i, j;
    for(i=0; i < 5; i++) {
        pair<int, int> p = make_pair(rand()%100, rand()%100 );
        pq.push(p );
        cout << "par: " << p.first << ", " << p.second << endl;
    }
    cout << "\nTamanho da fila de prioridade:" << pq.size() << endl;

    while(!pq.empty()) {
        cout << "par: " << pq.top().first << " " << pq.top().second << endl;
        pq.pop();
    }
}
```

Saída:

par: 86,83
par: 15,77
par: 35,93
par: 92,86
par: 21,49

Tamanho da fila de prioridade:5

par: **92** 86
par: **86** 83
par: **35** 93
par: **21** 49
par: **15** 77

Fila de prioridade - par de dados (**Second**)

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std; // priority_queue -> std

bool cmp(pair<int,int> a, pair<int,int> b) { return a.second < b.second || a.second == b.second && a.first < b.first; }

int main() {
    // heap min
    priority_queue<pair<int,int>, vector<pair<int,int>>, bool(*)>(pair<int,int>,pair<int,int>)> pq(cmp);

    for( int i=0; i < 5; i++) {
        pair<int, int> p = make_pair(rand()%100, rand()%100 );
        pq.push(p );
        cout << "par: " << p.first << ", " << p.second << endl;
    }
    cout << "\nTamanho da fila de prioridade:" << pq.size() << endl;
    while(!pq.empty()) {
        cout << "par: " << pq.top().first << " " << pq.top().second << endl;
        pq.pop();
    }
}
```

Saída:

par: 86,83
par: 15,77
par: 35,93
par: 92,86
par: 21,49

Tamanho da fila de prioridade:5
par: 35 **93**
par: 92 **86**
par: 86 **83**
par: 15 **77**
par: 21 **49**

Fila de prioridade - priority_queue

Referências

https://cplusplus.com/reference/queue/priority_queue/priority_queue/

Siang Wun Song. Fila de Prioridade. MAC 5710 - Estruturas de Dados. IME/USP 2008

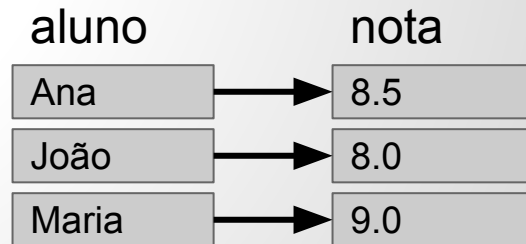


MAPA DE PARES

3 grupo

Mapa de pares (Maps)

- Os mapas são containers associativos que armazenam elementos formados por uma combinação de uma chave e um valor mapeado, seguindo ordem crescente pela chave.
- Cada chave deve ser um valor único.
- Internamente, os elementos em um mapa são sempre ordenados por sua chave



Mapa de pares (Maps)

```
#include <iostream>

#include <map>

using namespace std; // map -> std

int main() {
    map<string,float> mapa;
    for(int i=0; i < 5; i++) {
        string nome;
        float nota;
        cin >> nome >> nota;
        mapa[nome] = nota;
    }
    for(map<string,float>::iterator it = mapa.begin(); it != mapa.end(); it++)
        cout << it->first << " - " << it->second << endl;
}
```

Entrada

Ana 10
Jose 9
Maria 7.5
Joao 7.5
Noe 8

Saída:

Ana - 10
Joao - 7.5
Jose - 9
Maria - 7.5
Noe - 8

Mapa de pares (Maps)

```
#include <map>
```

Biblioteca necessária para o uso de mapa de pares.

```
map<TIPO_CHAVE, TIPO_ELEMENTO> m;
```

TIPO_CHAVE e TIPO_ELEMENTO podem receber qualquer tipo da linguagem: int, char, double, vector<float>, ...

```
m.clear();
```

Remove todos os elementos do mapa.

```
m[b] = a;
```

Atribui o valor **a** para a chave **b**.

```
a = m[b];
```

Ao acessar um índice que **não existe**, ele passa a existir com o valor 0 (para inteiros). Cuidado ao utilizar o valor 0 quando usar map.

```
it = m.find(CHAVE);
```

Se CHAVE (TIPO_CHAVE) existe, retorna um iterator para ele, caso contrário retorna `m.end()`.

```
m.count(CHAVE);
```

Se CHAVE existe, retorna um 1, caso contrário retorna 0.

```
it = m.begin();
```

Retorna iterator para o elemento inicial do mapa.

```
it = m.end();
```

Retorna iterator para o fim do mapa, **e não contém um elemento!**

```
m.erase(ELEMENTO);
```

Remove ELEMENTO (TIPO_chave), caso ele exista no mapa.

```
m.erase(it);
```

Remove elemento apontado pelo iterator it.

```
m.size();
```

Retorna a quantidade de elementos do mapa.

```
it->first
```

Acessa a chave (`it->first`) de um elemento no mapa.

```
it->second
```

Acessa o conteúdo (`it->second`) de um elemento no mapa.

```
m.insert( PAR_DE_DADOS );
```

Adiciona PAR DE DADOS (CHAVE, ELEMENTO) do tipo `<TIPO_CHAVE, TIPO_ELEMENTO>`, **caso a chave ainda não exista!**

Exemplo - Mapa de pares

```
#include <iostream>
#include <map>
#include <iterator>
#include <utility>
using namespace std; // map -> std

int main() {
    map<char, int> m, m2;
    map<char, int>::iterator it;
    pair<char, int> par('c', 50);
    m['a'] = 150;
    m['b'] = 200;
    m.insert(par);
    m2.insert(m.begin(), m.find('c'));
    it = m.end();
    it--;
    cout << "Último elemento do mapa m: chave(" << it->first << ") - valor: " << it->second;
}
```

Saída:

Último elemento do mapa m: chave(c) - valor: 50

Mapa de pares (Maps)

Saiba mais em:

<https://cplusplus.com/reference/map/map/>